SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA0823 – PYTHON PROGRAMMING

CO1 AT1

Coding Task Report

Submitted by: D.S.Tharuniya Priya

Register Number: 192511317

Department : Computer Science and Engineering

Course: CSA0823 – Python Programming

GitHub Repository: https://github.com/tharu-ui/CSA0823---Python-Programing

Submission Date: 21 October 2026

TASK 1 – PROBLEM ANALYSIS (4 MARKS)

Aim

To design a Python program that calculates the rate per litre and the payment due to a farmer using milk quantity, fat percentage and solids-not-fat (SNF) percentage.

Problem Description

A village milk co-operative needs to calculate payments consistently according to a rate card. The base rate is Rs 30.00 per litre for milk containing 3.5% fat and 8.5% SNF. Each 0.1 percentage-point fat step changes the rate by Rs 0.40 per litre, and each 0.1 percentage-point SNF step changes it by Rs 0.30 per litre.

Inputs, Processing and Outputs

Element	Description
Input	Milk quantity in litres, fat percentage and SNF percentage.
Processing	Validate values, calculate fat and SNF steps, calculate the rate and calculate the payment.
Output	Rate per litre and total payment, displayed to two decimal places.
Validation	Reject invalid text, non-positive quantity and measurements outside the limits specified by the assessment.

Formulae

Fat steps = round((fat − 3.5) / 0.1)

SNF steps = round((SNF − 8.5) / 0.1)

Rate = 30.00 + (fat steps × 0.40) + (SNF steps × 0.30)

Payment = litres × rate, rounded to two decimal places.

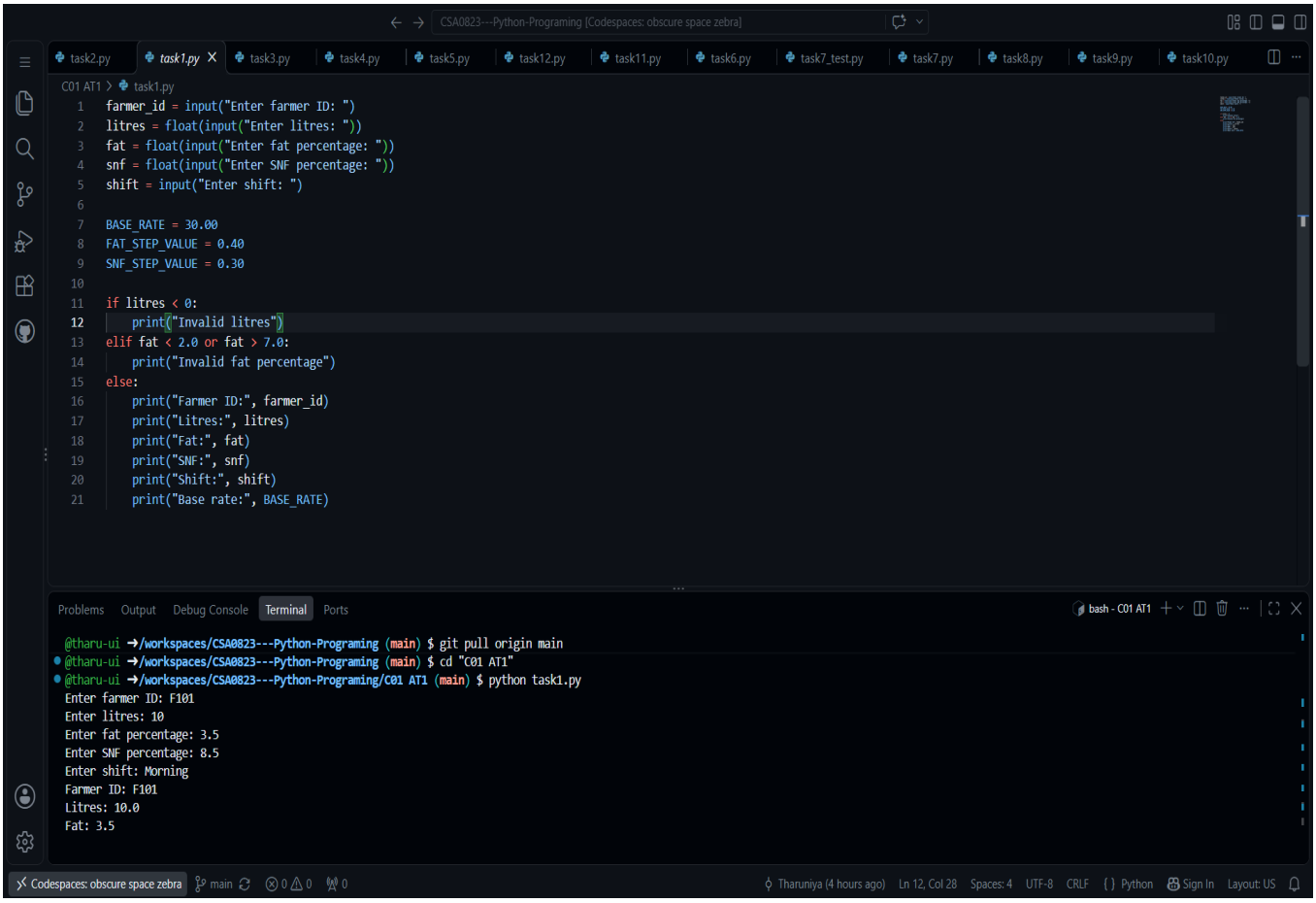
Algorithm

1. Start the program.
2. Read the milk quantity, fat percentage and SNF percentage.

3. Validate that the inputs are numeric and within the permitted limits.
4. Calculate the fat steps and SNF steps from the standard values.
5. Calculate the rate per litre using the rate card.
6. Multiply the quantity by the rate to obtain the payment.
7. Display the rate and payment to two decimal places.
8. Stop the program.

Complexity Analysis

For one entry, a fixed number of arithmetic operations is performed. The time complexity is O(1), and the auxiliary space complexity is O(1). For n entries, processing all entries takes O(n) time. A running total uses O(1) additional space, while storing all entries for later review uses O(n) space.


CSA0823---Python-Programing [Codespaces: obscure space zebra]
task2.py
task1.py
task3.py
task4.py
task5.py
task12.py
task11.py
task6.py
task7_test.py
task7.py
task8.py
task9.py
task10.py
C01 AT1 > task1.py
1 farmer_id = input("Enter farmer ID: ")
2 litres = float(input("Enter litres: "))
3 fat = float(input("Enter fat percentage: "))
4 snf = float(input("Enter SNF percentage: "))
5 shift = input("Enter shift: ")
6
7 BASE_RATE = 30.00
8 FAT_STEP_VALUE = 0.40
9 SNF_STEP_VALUE = 0.30
10
11 if litres < 0:
12 print("Invalid litres")
13 elif fat < 2.0 or fat > 7.0:
14 print("Invalid fat percentage")
15 else:
16 print("Farmer ID:", farmer_id)
17 print("Litres:", litres)
18 print("Fat:", fat)
19 print("SNF:", snf)
20 print("Shift:", shift)
21 print("Base rate:", BASE_RATE)
Problems
Output
Debug Console
Terminal
Ports
bash - C01 AT1
@tharu-ui ➜/workspaces/CSA0823---Python-Programing (main) $ git pull origin main
@tharu-ui ➜/workspaces/CSA0823---Python-Programing (main) $ cd "C01 AT1"
@tharu-ui ➜/workspaces/CSA0823---Python-Programing/C01 AT1 (main) $ python task1.py
Enter farmer ID: F101
Enter litres: 10
Enter fat percentage: 3.5
Enter SNF percentage: 8.5
Enter shift: Morning
Farmer ID: F101
Litres: 10.0
Fat: 3.5
Codespaces: obscure space zebra
main
Tharuniya (4 hours ago)
Ln 12, Col 28
Spaces: 4
UTF-8
CRLF
Python
Sign In
Layout: US

TASK 2 – VALUES AND TYPES (4 MARKS)

Objective

This task compares floating-point arithmetic with decimal arithmetic and integer paise to understand the importance of exact monetary calculations.

Program: Float and Decimal Totals

```
from decimal import Decimal

payments = ["33.40"] * 1000 + ["0.10"] * 3

def total_float(items):
    """Return the sum using float."""
    total = 0.0
    for amount in items:
        total += float(amount)
    return total

def total_decimal(items):
    """Return the exact decimal sum."""
    total = Decimal("0.00")
    for amount in items:
        total += Decimal(amount)
    return total

print("float :", total_float(payments))
print("Decimal:", total_decimal(payments))
print(0.1 + 0.2 == 0.3,
      Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))
```

Expected Output

```
float : 33400.30000000061
```

Decimal: 33400.30

False True

Explanation

Python float values use binary floating-point representation. Decimal fractions such as 0.1 and 0.2 generally cannot be represented exactly in binary, so small approximation errors may accumulate. Decimal values constructed directly from strings preserve decimal inputs such as '33.40' and '0.10' exactly for these calculations.

Effect of Converting Float to Decimal

total += Decimal(float(amount))

When the input string is first converted to float, its small binary approximation error is introduced. Decimal(float(amount)) then preserves that approximate value; it cannot reconstruct the original exact decimal amount. Decimal(amount), where amount is the original string, is the preferred approach for decimal monetary input.

Alternative Methods and Complexity

Method	Time per addition	Accumulator space
float	O(1)	O(1)
Decimal from strings	O(1) for bounded-size values	O(1)
Integer paise	O(1)	O(1)
Round after each float addition	O(1)	O(1)

Memory Measurement

import sys

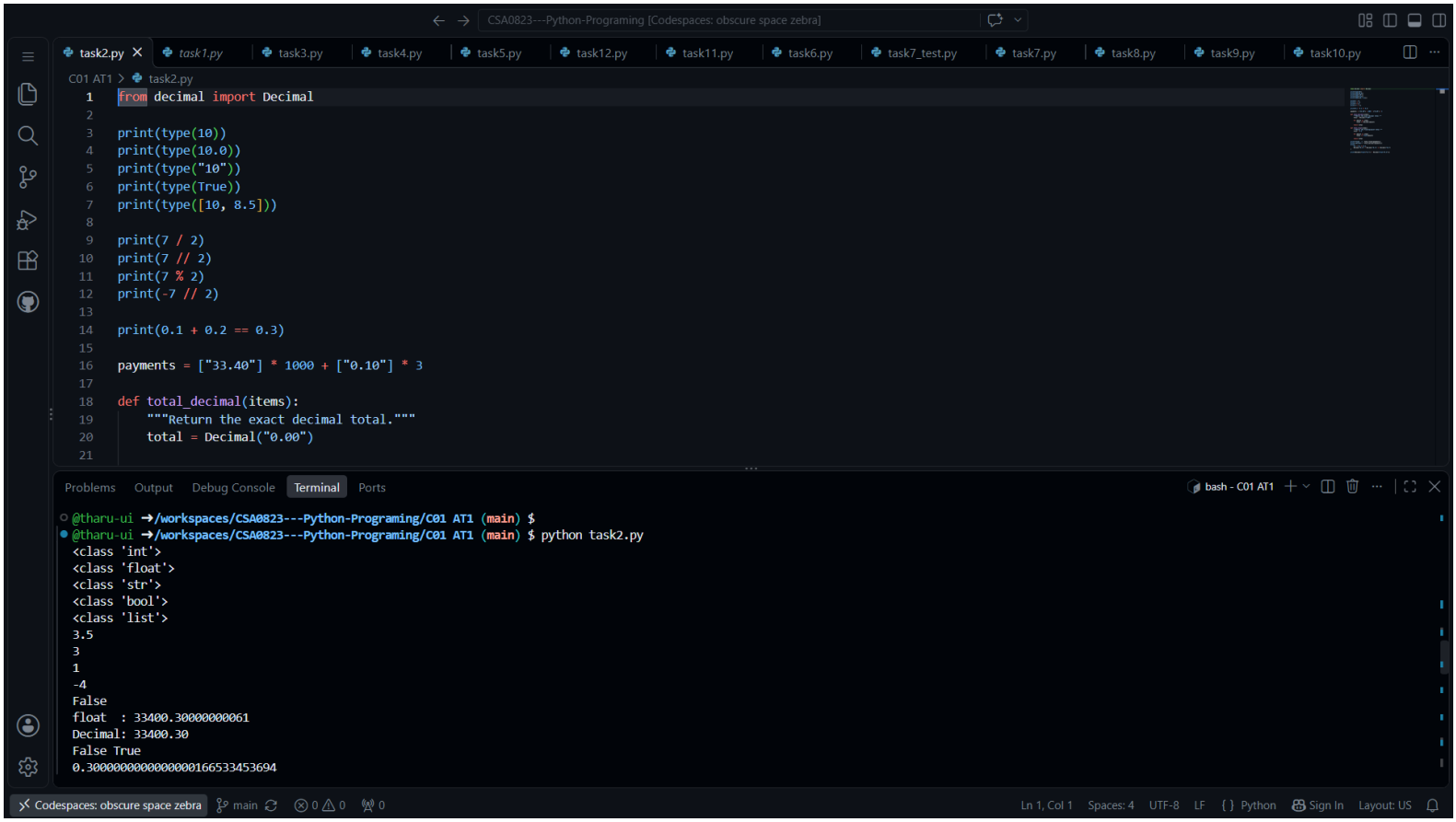
from decimal import Decimal

```
print("float:", sys.getsizeof(0.1))
print("Decimal:", sys.getsizeof(Decimal("0.1")))
print("integer:", sys.getsizeof(10))
```

Integer-Paise Alternative

```
total_paise = 0
for amount in payments:
    rupees, paise = amount.split(".")
    total_paise += int(rupees) * 100 + int(paise)
print(f"Integer paise total: Rs {total_paise / 100:.2f}")
```

The integer accumulator gives 3,340,030 paise, equivalent to Rs 33,400.30.


CSA0823---Python-Programing [Codespaces: obscure space zebra]
task2.py
task1.py
task3.py
task4.py
task5.py
task12.py
task11.py
task6.py
task7_test.py
task7.py
task8.py
task9.py
task10.py
C01 AT1 > task2.py
1 from decimal import Decimal
2
3 print(type(10))
4 print(type(10.0))
5 print(type("10"))
6 print(type(True))
7 print(type([10, 8.5]))
8
9 print(7 / 2)
10 print(7 // 2)
11 print(7 % 2)
12 print(-7 // 2)
13
14 print(0.1 + 0.2 == 0.3)
15
16 payments = ["33.40"] * 1000 + ["0.10"] * 3
17
18 def total_decimal(items):
19 """Return the exact decimal total."""
20 total = Decimal("0.00")
21
Problems
Output
Debug Console
Terminal
Ports
bash - C01 AT1
@tharu-ui ➜/workspaces/CSA0823---Python-Programing/C01 AT1 (main) $
@tharu-ui ➜/workspaces/CSA0823---Python-Programing/C01 AT1 (main) $ python task2.py
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'list'>
3.5
3
1
-4
False
float : 33400.30000000061
Decimal: 33400.30
False True
0.30000000000000016653345369
Codespaces: obscure space zebra
main
Ln 1, Col 1
Spaces: 4
UTF-8
LF
{ } Python
Sign In
Layout: US

TASK 3 – VARIABLES AND CONSTANTS (3 MARKS)

Program

```
BASE_RATE = 30.00
STANDARD_FAT = 3.5
STANDARD_SNF = 8.5
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
STEP_SIZE = 0.1

fat = float(input("Enter fat percentage: "))
snf = float(input("Enter SNF percentage: "))

fat_steps = round((fat - STANDARD_FAT) / STEP_SIZE)
snf_steps = round((snf - STANDARD_SNF) / STEP_SIZE)

rate = BASE_RATE + fat_steps * FAT_STEP_VALUE
rate += snf_steps * SNF_STEP_VALUE

print("Fat steps:", fat_steps)
print("SNF steps:", snf_steps)
print(f"Rate per litre: Rs {rate:.2f}")
```

Naming Rules and Conventions

- A Python identifier can contain letters, digits and underscores, but it cannot begin with a digit.
- Names are case-sensitive, so fat, Fat and FAT are different names.
- Meaningful names such as fat_steps improve readability.
- Constants are conventionally written in uppercase, for example BASE_RATE.

Why Python Has No True Constants

Python does not prevent a programmer from reassigning a name written in uppercase. Uppercase is a convention that signals a value should not normally be changed during execution.

Dynamic Typing

```
value = 10
print(type(value))
value = "ten"
print(type(value))
```

The first type is int and the second type is str. Python determines the type from the object assigned to the name, and a name can later refer to an object of another type.

Multiple Assignment and Swapping

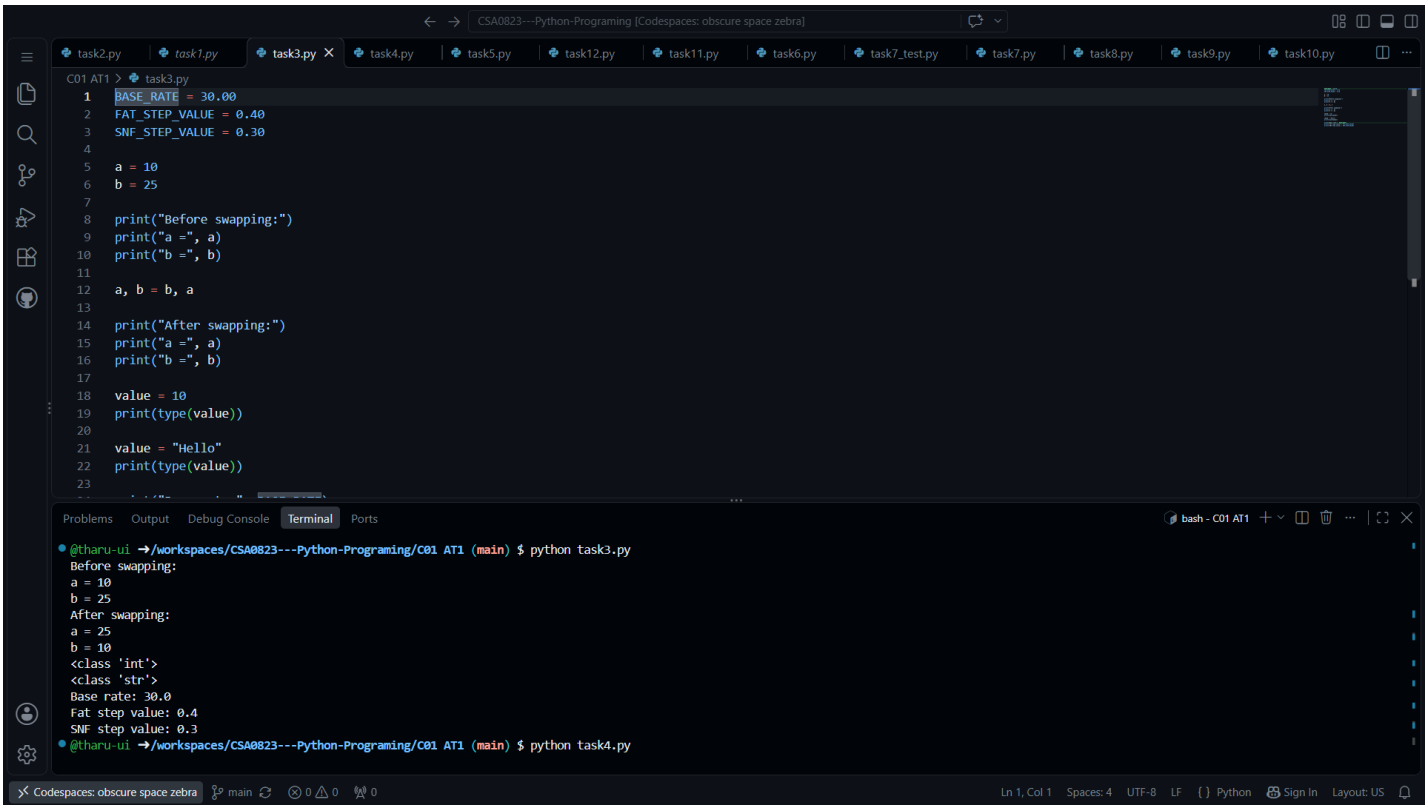
```
a = 10
b = 25
a, b = b, a
print(a, b)  # 25 10
```

Alternative Concept and Complexity

Approach	Rate revision	Runtime and space
Magic numbers	Every expression containing the rate may need editing (k locations).	Usually O(1) per calculation.
Named constants	Change the relevant constant definition once.	O(1) per calculation.
RATE_CARD dictionary	Change a key-value entry in one dictionary.	Average O(1) lookup; O(t) space for t stored values.

```
RATE_CARD = {
    "base": 30.00,
    "fat": 0.40,
    "snf": 0.30
}
```

Named constants are sufficient for one fixed rate card. A dictionary becomes useful when the program must select among several named rate cards.



TASK 4 – WRITE CORRECT EXPRESSIONS (4 MARKS)

Expressions

fat_steps = round((fat - 3.5) / 0.1)

snf_steps = round((snf - 8.5) / 0.1)

rate = 30.00 + fat_steps * 0.40 + snf_steps * 0.30

rate = round(rate, 2)

amount = round(litres * rate, 2)

Operator Precedence and Associativity

- Parentheses are evaluated first.
- Exponentiation ** has higher precedence than multiplication and addition. For example, -2**2 is -(2**2), which equals -4; (-2)**2 equals 4.
- Multiplication and division are evaluated before addition and subtraction.
- Multiplication and division at the same precedence level are evaluated from left to right.

```
print(2 + 3 * 4)    # 14
print((2 + 3) * 4)  # 20
print(-2**2)        # -4
print((-2)**2)      # 4
```

Unexpected Floating-Point Results

Expression	Result	Explanation
(4.2 - 3.5) / 0.1	7.000000000000001	Binary floating-point approximations affect the division.
0.1 + 0.2	0.30000000000000004	The decimal fractions are not exactly represented in binary.

round(2.675, 2)	2.67	The stored float is slightly below the exact decimal value.

round() is used to bring the computed step count to the nearest whole step, but Python uses ties-to-even rounding for exact halfway cases. A real payment system must document the required rounding policy.

round(), int() and math.floor()

```
import math
print(round(-2.9))       # -3
print(int(-2.9))         # -2
print(math.floor(-2.9))  # -3
```

round() chooses the nearest integer, int() truncates towards zero, and math.floor() rounds towards negative infinity.

Fat	Approximate steps	round()	int()	floor()
4.2	7.000000000000001	7	7	7
3.1	-3.999999999999999	-4	-3	-4

If SNF is 8.5%, the rates are Rs 32.80 for fat 4.2 under all three methods. For fat 3.1, round() and floor() give Rs 28.40, while int() gives Rs 28.80 because it truncates the negative step count towards zero.

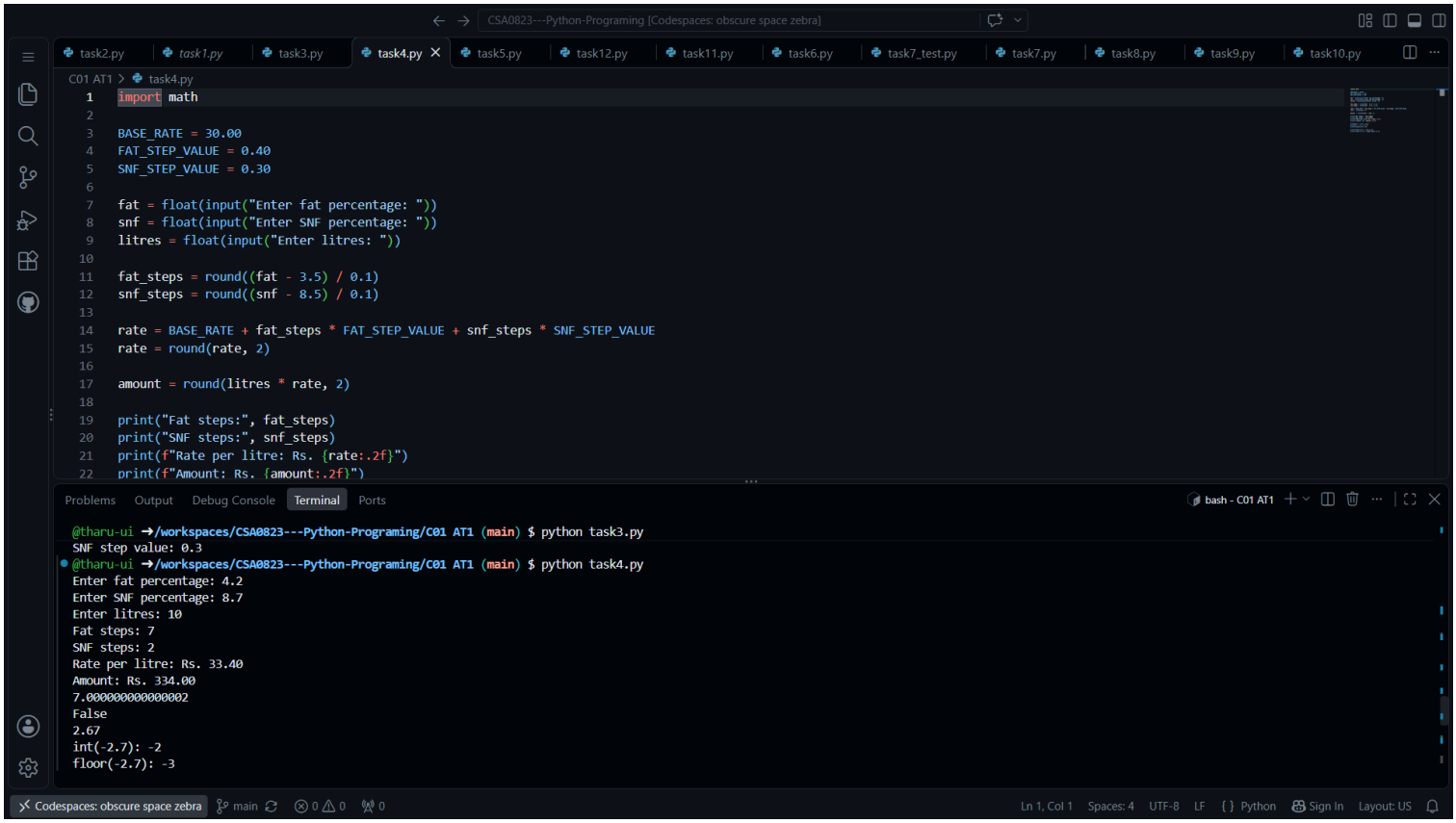
Comparison Chaining and Boolean Operators

```
valid_fat = 2.0 <= fat <= 7.0
valid_entry = litres > 0 and valid_fat and 0.0 <= snf <= 12.0
```

Comparison chaining checks both bounds without repeating the variable. The Boolean operator and requires all listed conditions to be true. In this report, the validation limits are fat from 2.0% to 7.0% and SNF from 0.0% to 12.0%.

Complexity

The arithmetic, rounding and comparisons use O(1) time and O(1) auxiliary space for one entry.



TASK 5 – READ INPUT AND CONVERT TYPES (3 MARKS)

Input Program

```
def read_number(prompt):
    """Read a valid floating-point value."""
    while True:
        text = input(prompt).strip()
        try:
            return float(text)
        except ValueError:
            print("Invalid input. Enter a number such as 8.5.")

litres = read_number("Enter litres: ")
fat = read_number("Enter fat percentage: ")
snf = read_number("Enter SNF percentage: ")

if litres <= 0:
    print("Milk quantity must be positive.")
elif not (2.0 <= fat <= 7.0):
    print("Fat percentage is outside the allowed range.")
elif not (0.0 <= snf <= 12.0):
    print("SNF percentage is outside the allowed range.")
else:
    print(f"Litres: {litres:8.2f}")
    print(f"Fat:    {fat:8.2f}")
    print(f"SNF:    {snf:8.2f}")
```

Input Conversion

input() always returns text as a string. int() converts suitable integer text, while float() converts suitable numeric text to a floating-point value. Invalid conversions such as float('eight') raise ValueError.

Input text	Result
"8.5"	Accepted as 8.5.
" 8.5 "	Accepted after surrounding whitespace is removed.
"8,5"	Rejected by float() in the standard decimal format.
"eight"	Rejected because it is not a numeric literal.

String Checks versus try/except

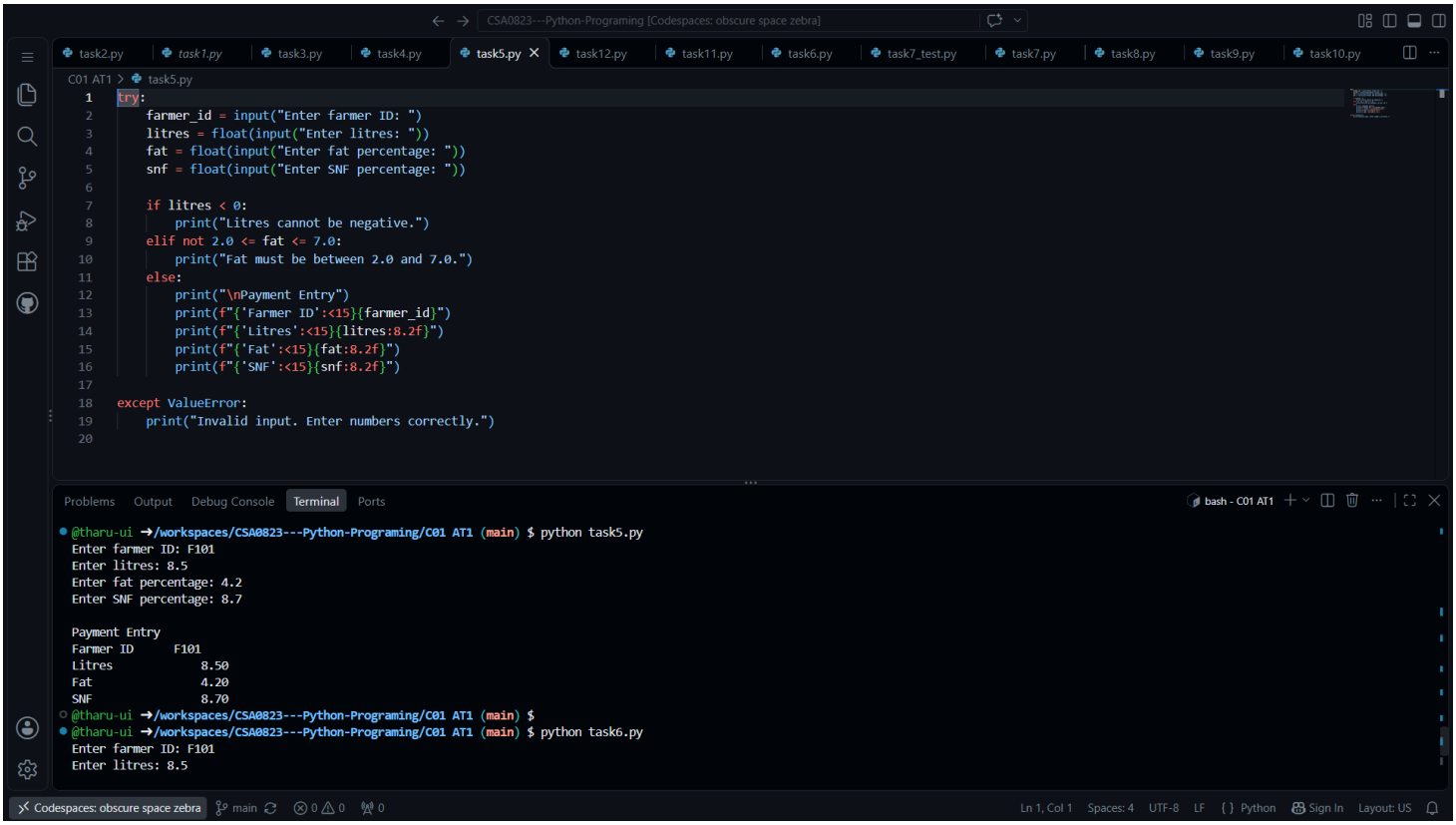
Methods such as isnumeric() are not suitable for validating all decimal strings because '8.5' contains a decimal point and is not considered numeric by isnumeric(). Using try/except ValueError around float(text) is clear and directly tests whether Python can parse the number.

For ordinary bounded-length input, each conversion takes O(1) time and O(1) auxiliary space. Exception handling has additional overhead for invalid input, but is usually adequate for clerk-entered values.

Aligned Output

```
amount = 283.9
print(f"{amount:8.2f}")
```

The format specification 8.2f reserves a field width of eight characters and displays two digits after the decimal point. This helps align values in printed reports.


CSA0823---Python-Programing [Codespaces: obscure space zebra]
task2.py
task1.py
task3.py
task4.py
task5.py
task12.py
task11.py
task6.py
task7_test.py
task7.py
task8.py
task9.py
task10.py
C01 AT1 > task5.py
try:
farmer_id = input("Enter farmer ID: ")
litres = float(input("Enter litres: "))
fat = float(input("Enter fat percentage: "))
snf = float(input("Enter SNF percentage: "))
if litres < 0:
print("Litres cannot be negative.")
elif not 2.0 <= fat <= 7.0:
print("Fat must be between 2.0 and 7.0.")
else:
print("\nPayment Entry")
print(f"{'Farmer ID':<15}{farmer_id}")
print(f"{'Litres':<15}{litres:8.2f}")
print(f"{'Fat':<15}{fat:8.2f}")
print(f"{'SNF':<15}{snf:8.2f}")
except ValueError:
print("Invalid input. Enter numbers correctly.")
Problems
Output
Debug Console
Terminal
Ports
bash - C01 AT1
@tharu-ui →/workspaces/CSA0823---Python-Programing/C01 AT1 (main) $ python task5.py
Enter farmer ID: F101
Enter litres: 8.5
Enter fat percentage: 4.2
Enter SNF percentage: 8.7
Payment Entry
Farmer ID F101
Litres 8.50
Fat 4.20
SNF 8.70
@tharu-ui →/workspaces/CSA0823---Python-Programing/C01 AT1 (main) $
@tharu-ui →/workspaces/CSA0823---Python-Programing/C01 AT1 (main) $ python task6.py
Enter farmer ID: F101
Enter litres: 8.5
Codespaces: obscure space zebra
main
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Sign In
Layout: US

TASK 6 – ORGANISE THE LOGIC INTO FUNCTIONS (4 MARKS)

Functions and Code Structure

The calculator is divided into rate_per_litre(), payout() and ten_day_total(). Each function performs one task, making the program easier to read, test and maintain.

Function	Parameters	Return value
rate_per_litre(fat, snf)	Fat and SNF percentages.	Rate per litre.
payout(litres, fat, snf)	Quantity, fat and SNF.	Payment for one entry.
ten_day_total(entries)	A collection of entries.	Total payment.

Return versus Print

```
def square(number):
    """Return the square of a number."""
    return number * number

result = square(4)
print(result)
```

The function returns 16 to its caller, which can store or reuse it. A function that only prints a result does not return that printed value; its default return value is None.

Scope and Constants

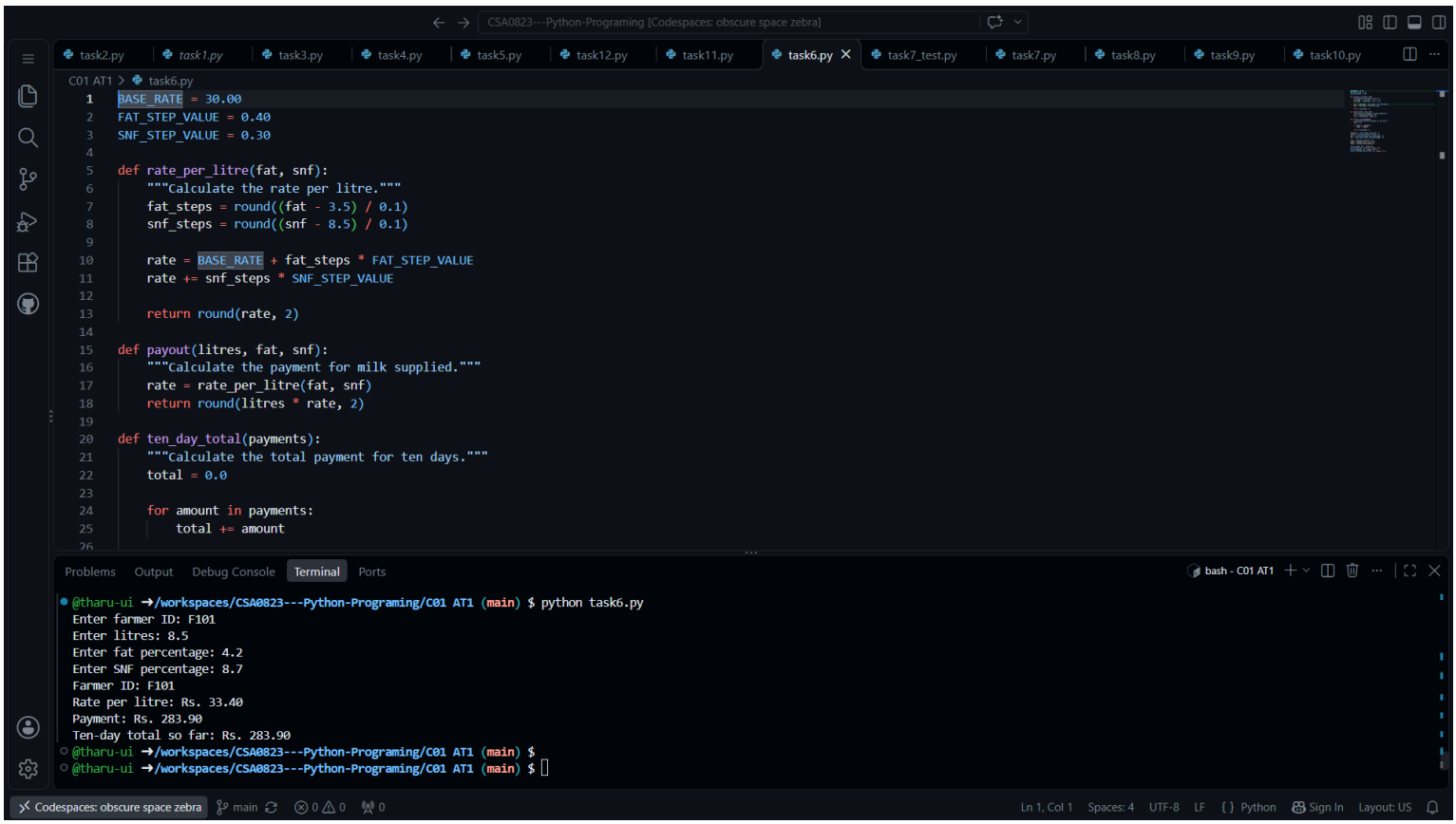
Names created inside a function, such as fat_steps and rate, are local to that function. Module-level constants can be read from functions without declaring them global. Rate constants should not be modified inside calculation functions.

Docstrings

A docstring is a descriptive string placed at the beginning of a function. It explains the function's purpose and can be accessed using documentation tools.

Alternative Concept and Complexity

A long script without functions may repeat formulas and make corrections difficult. Lambda expressions are useful for small expressions but are less suitable for multi-step calculations that need docstrings and validation. Function calls add a small overhead; processing n entries remains O(n) time. Storing all entries uses O(n) space, while a running total alone uses O(1) additional space.



TASK 7 – BUILD A PYTHON MODULE (4 MARKS)

Module: milkrate.py

```
"""Rate card of the village milk co-operative."""

BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
STANDARD_FAT = 3.5
STANDARD_SNF = 8.5
STEP_SIZE = 0.1

def rate_per_litre(fat, snf):
    """Return the rate per litre rounded to two decimals."""
    fat_steps = round((fat - STANDARD_FAT) / STEP_SIZE)
    snf_steps = round((snf - STANDARD_SNF) / STEP_SIZE)
    rate = BASE_RATE + fat_steps * FAT_STEP_VALUE
    rate += snf_steps * SNF_STEP_VALUE
    return round(rate, 2)

def payout(litres, fat, snf):
    """Return the payment rounded to two decimals."""
    return round(litres * rate_per_litre(fat, snf), 2)

if __name__ == "__main__":
    for litres, fat, snf in [
        (10, 3.5, 8.5),
        (8.5, 4.2, 8.7),
        (12, 3.1, 8.3)
    ]:
        rate = rate_per_litre(fat, snf)
        pay = payout(litres, fat, snf)
        print(
            f"{litres:5} L fat {fat} snf {snf} "
```

```
        f"rate {rate:6.2f} pay {pay:8.2f}"
    )
```

Expected Output

```
 10 L fat 3.5 snf 8.5 rate  30.00 pay   300.00
8.5 L fat 4.2 snf 8.7 rate  33.40 pay   283.90
 12 L fat 3.1 snf 8.3 rate  27.80 pay   333.60
```

Separate Importing Script

```
import milkrate

rate = milkrate.rate_per_litre(4.2, 8.7)
payment = milkrate.payout(8.5, 4.2, 8.7)

print("Rate:", rate)
print("Payment:", payment)
```

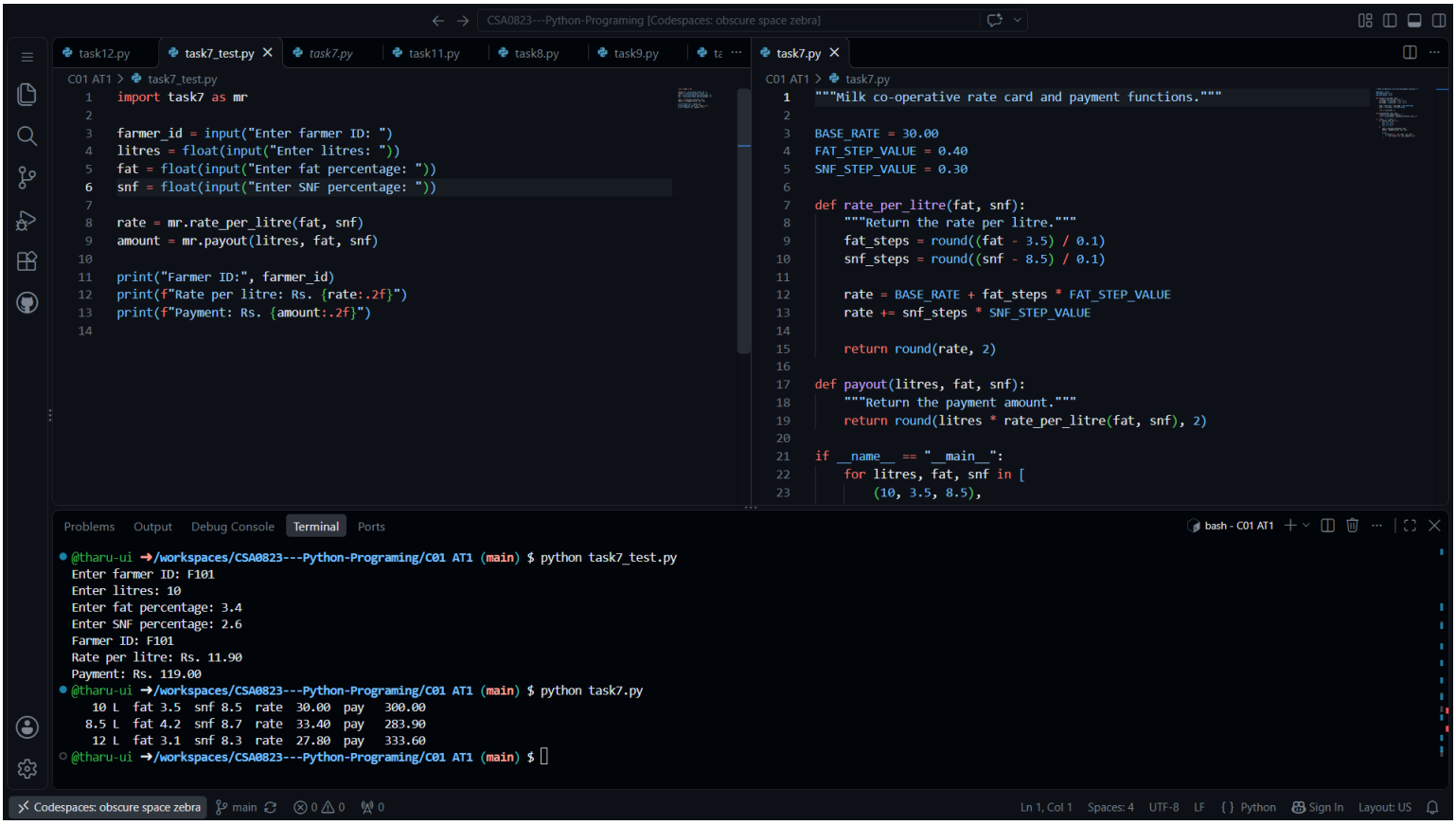
The import script prints Rate: 33.4 and Payment: 283.9. The demonstration table is not printed on import because the __name__ == '__main__' condition is false when the file is imported. Without the guard, executable statements at module level would run during import.

Import Forms and Search Path

- import milkrate: access functions using milkrate.payout().
- from milkrate import payout: access payout() directly.
- import milkrate as mr: use the alias mr, for example mr.payout().
- Python searches locations listed in sys.path, commonly including the script directory and standard library paths.
- Loaded modules are normally cached in sys.modules, avoiding repeated execution on ordinary subsequent imports.

Alternative and Complexity

Copying the functions into every script duplicates code and makes rate revisions harder. A shared module centralizes the implementation. Importing has a one-time loading cost; each rate or payout calculation remains O(1).



TASK 8 – PLAN THE ALGORITHM BEFORE CODING (4 MARKS)

Algorithmic Problem-Solving Steps

1.Understand the problem by identifying inputs, outputs, limits and the rate card.

2.Plan the formulas, validation, pseudocode and flowchart.

3.Write the Python implementation using the planned steps.

4.Test normal values, boundaries and invalid values, then compare expected and actual results.

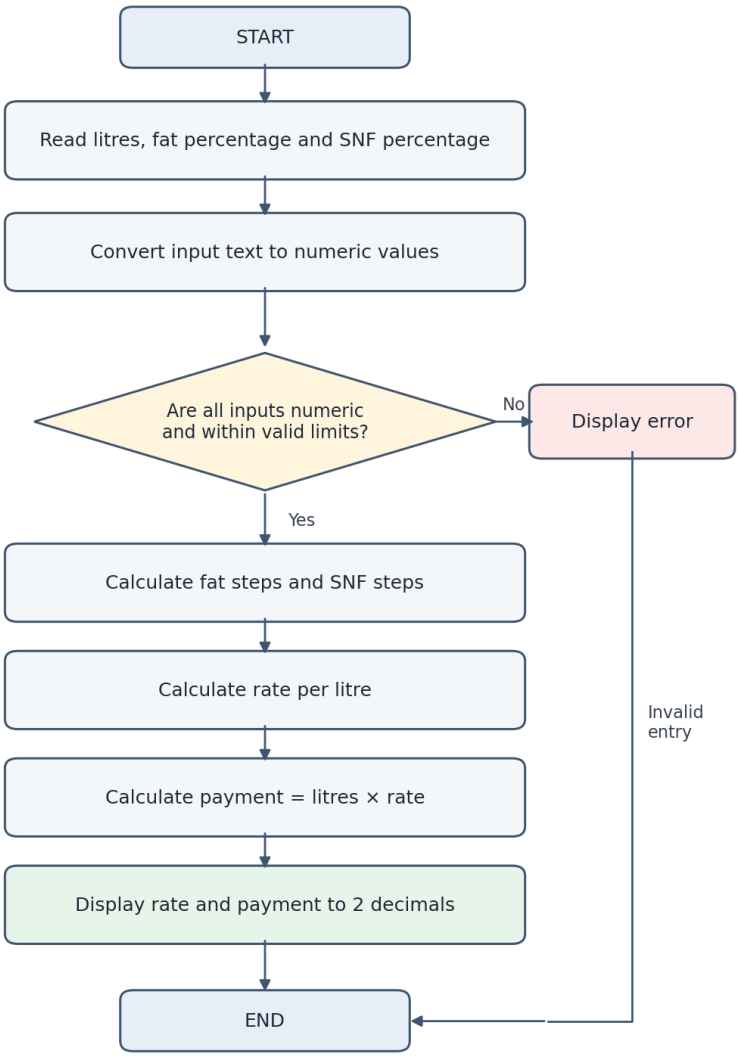
Pseudocode

```
ALGORITHM MilkPayment
CONSTANT BASE_RATE = 30.00
CONSTANT FAT_STEP_VALUE = 0.40
CONSTANT SNF_STEP_VALUE = 0.30
CONSTANT STANDARD_FAT = 3.5
CONSTANT STANDARD_SNF = 8.5
CONSTANT STEP_SIZE = 0.1

BEGIN
   INPUT litres, fat, snf
     IF litres <= 0 OR fat < 2.0 OR fat > 7.0
     OR snf < minimum_snf OR snf > maximum_snf THEN
      OUTPUT "Invalid entry"
   ELSE
      fat_steps = ROUND((fat - STANDARD_FAT) / STEP_SIZE)
      snf_steps = ROUND((snf - STANDARD_SNF) / STEP_SIZE)
      rate = BASE_RATE + fat_steps * FAT_STEP_VALUE
            + snf_steps * SNF_STEP_VALUE
      rate = ROUND(rate, 2)
      payment = ROUND(litres * rate, 2)
      OUTPUT rate, payment
   END IF
END
```

The pseudocode uses the same validation limits as the implementation: fat from 2.0% to 7.0% and SNF from 0.0% to 12.0%.

Flowchart for Validating an Entry



Hand Trace

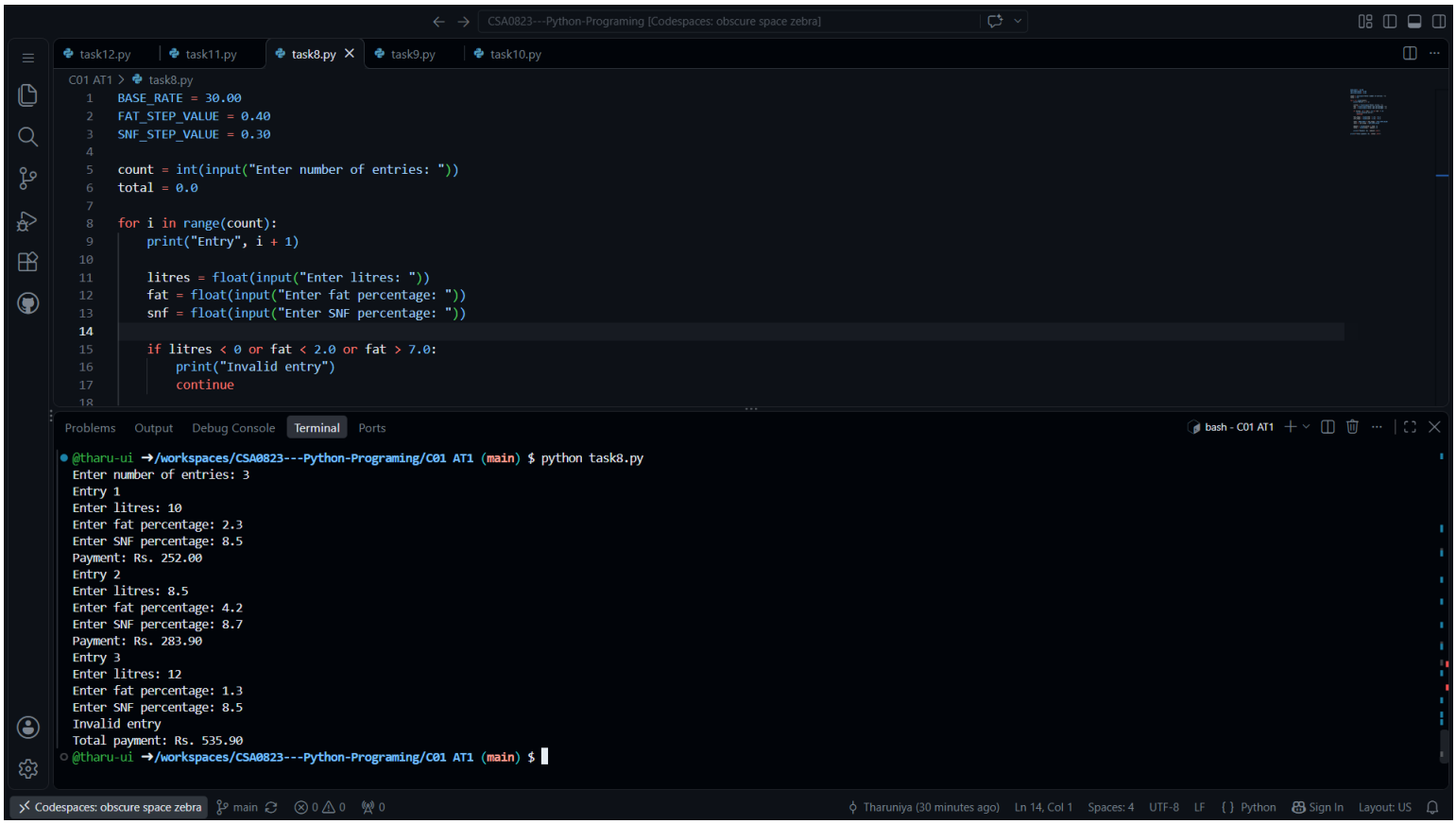
Entry	Litres	Fat	SNF	Rate	Payment
1	10	3.5	8.5	Rs 30.00	Rs 300.00
2	8.5	4.2	8.7	Rs 33.40	Rs 283.90

3	12	3.1	8.3	Rs 27.80	Rs 333.60
Total					Rs 917.50

The total is Rs 300.00 + Rs 283.90 + Rs 333.60 = Rs 917.50.

Store Entries versus Running Total

Approach	Time	Extra space	Use
Store all entries, then total	O(n)	O(n)	When records are needed for audit, review or correction.
Maintain running total	O(n)	O(1)	When only the final total is needed.


CSA0823---Python-Programing [Codespaces: obscure space zebra]
task12.py
task11.py
task8.py
task9.py
task10.py
C01 AT1 > task8.py
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
count = int(input("Enter number of entries: "))
total = 0.0
for i in range(count):
print("Entry", i + 1)
litres = float(input("Enter litres: "))
fat = float(input("Enter fat percentage: "))
snf = float(input("Enter SNF percentage: "))
if litres < 0 or fat < 2.0 or fat > 7.0:
print("Invalid entry")
continue
Problems
Output
Debug Console
Terminal
Ports
bash - C01 AT1
@tharu-ui ➜/workspaces/CSA0823---Python-Programing/C01 AT1 (main) $ python task8.py
Enter number of entries: 3
Entry 1
Enter litres: 10
Enter fat percentage: 2.3
Enter SNF percentage: 8.5
Payment: Rs. 252.00
Entry 2
Enter litres: 8.5
Enter fat percentage: 4.2
Enter SNF percentage: 8.7
Payment: Rs. 283.90
Entry 3
Enter litres: 12
Enter fat percentage: 1.3
Enter SNF percentage: 8.5
Invalid entry
Total payment: Rs. 535.90
@tharu-ui ➜/workspaces/CSA0823---Python-Programing/C01 AT1 (main) $
Codespaces: obscure space zebra
main
Tharuniya (30 minutes ago)
Ln 14, Col 1
Spaces: 4
UTF-8
LF
Python
Sign In
Layout: US

TASK 9 – IMPLEMENT AND TEST THE CALCULATOR (4 MARKS)

Financially Safer Implementation (save as milk_calculator.py)

The following implementation uses Decimal for monetary values and ROUND_HALF_UP as an explicit example rounding policy. The co-operative should confirm that this policy matches its official payment rules.

```
from decimal import Decimal, ROUND_HALF_UP, InvalidOperation

BASE_RATE = Decimal("30.00")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
STANDARD_FAT = Decimal("3.5")
STANDARD_SNF = Decimal("8.5")
STEP_SIZE = Decimal("0.1")

def money(value):
    """Round a monetary value to two decimal places."""
    return value.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)

def rate_per_litre(fat, snf):
    """Calculate the rate per litre."""
    fat_steps = int(((fat - STANDARD_FAT) / STEP_SIZE).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP))
    snf_steps = int(((snf - STANDARD_SNF) / STEP_SIZE).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP))
    rate = BASE_RATE + fat_steps * FAT_STEP_VALUE
    rate += snf_steps * SNF_STEP_VALUE
    return money(rate)

def payout(litres, fat, snf):
    """Calculate payment for a valid entry."""
    if litres <= 0:
        raise ValueError("Litres must be positive.")
```

```
    if not Decimal("2.0") <= fat <= Decimal("7.0"):
        raise ValueError("Fat must be between 2.0 and 7.0.")
    if not Decimal("0.0") <= snf <= Decimal("12.0"):
        raise ValueError("SNF must be between 0.0 and 12.0.")
    return money(litres * rate_per_litre(fat, snf))

def read_decimal(prompt):
    """Read a valid Decimal number."""
    while True:
        text = input(prompt).strip()
        try:
            return Decimal(text)
        except InvalidOperation:
            print("Invalid number. Try again.")

if __name__ == "__main__":
    litres = read_decimal("Enter litres: ")
    fat = read_decimal("Enter fat percentage: ")
    snf = read_decimal("Enter SNF percentage: ")
    try:
        rate = rate_per_litre(fat, snf)
        amount = payout(litres, fat, snf)
        print(f"Rate per litre: Rs {rate:.2f}")
        print(f"Amount payable: Rs {amount:.2f}")
    except ValueError as error:
        print("Error:", error)
```

Automated Assertions

```
from decimal import Decimal
from milk_calculator import rate_per_litre, payout

assert rate_per_litre(Decimal("3.5"), Decimal("8.5")) == Decimal("30.00")
```

```
assert rate_per_litre(Decimal("4.2"), Decimal("8.7")) == Decimal("33.40")
assert rate_per_litre(Decimal("3.1"), Decimal("8.3")) == Decimal("27.80")
assert rate_per_litre(Decimal("2.0"), Decimal("8.5")) == Decimal("24.00")
assert rate_per_litre(Decimal("7.0"), Decimal("8.5")) == Decimal("44.00")
assert payout(Decimal("10"), Decimal("3.5"), Decimal("8.5")) == Decimal("300.00")
assert payout(Decimal("8.5"), Decimal("4.2"), Decimal("8.7")) == Decimal("283.90")
assert payout(Decimal("12"), Decimal("3.1"), Decimal("8.3")) == Decimal("333.60")

for litres, fat, snf in [
    (Decimal("-1"), Decimal("3.5"), Decimal("8.5")),
    (Decimal("10"), Decimal("8.0"), Decimal("8.5"))
]:
    try:
        payout(litres, fat, snf)
    except ValueError:
        pass
    else:
        raise AssertionError("Invalid input was not rejected")

print("All tests passed.")
```

Test Table

Test	Input / condition	Expected result	Actual result
T1	10 L, fat 3.5, SNF 8.5	Rs 300.00	Rs 300.00
T2	8.5 L, fat 4.2, SNF 8.7	Rs 283.90	Rs 283.90
T3	12 L, fat 3.1, SNF 8.3	Rs 333.60	Rs 333.60
T4	Fat 2.0, SNF 8.5	Rate Rs 24.00	Rate Rs 24.00
T5	Fat 7.0, SNF 8.5	Rate Rs 44.00	Rate Rs 44.00

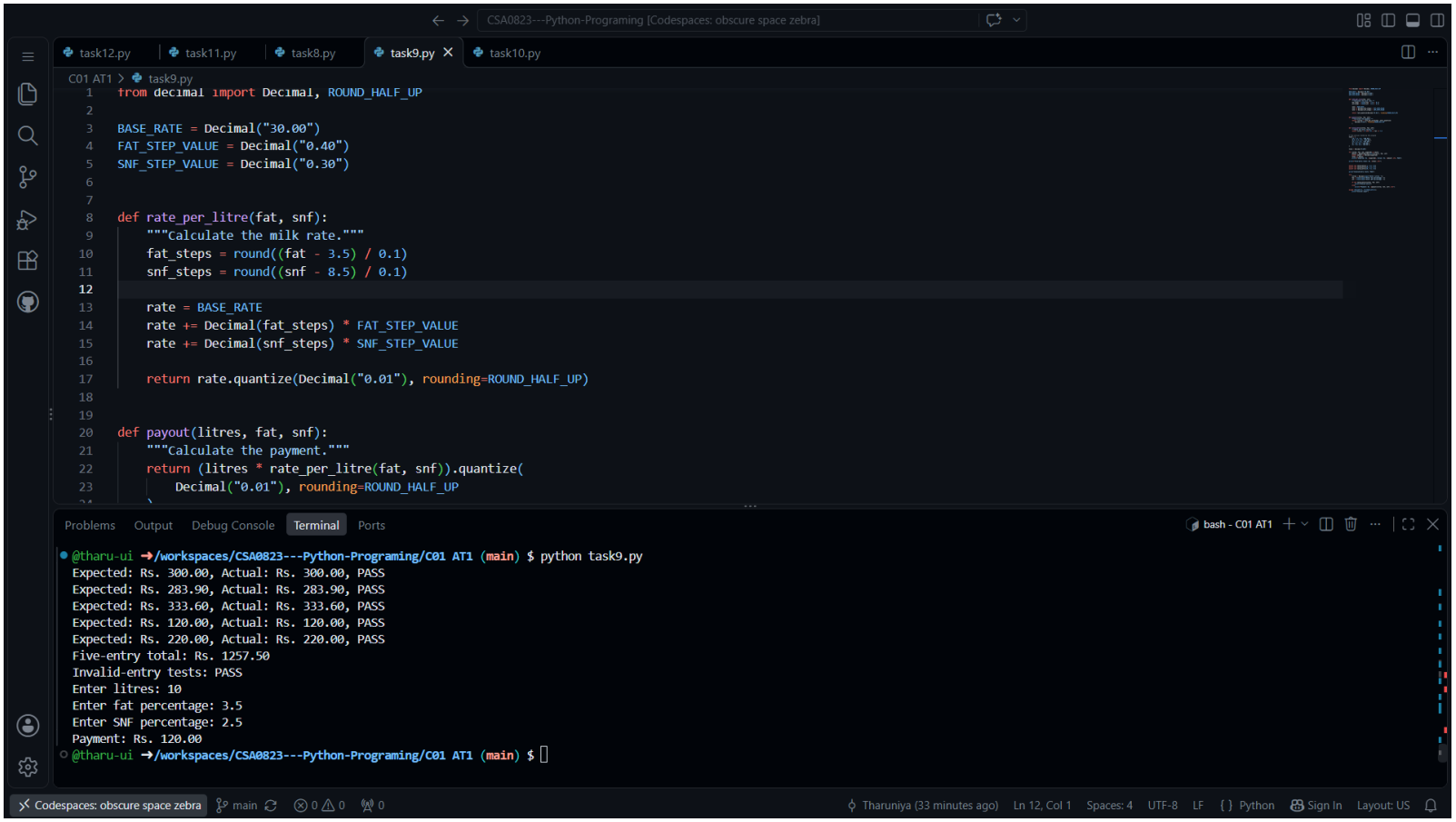
T6	Negative quantity	ValueError / rejection	Rejected (ValueError)
T7	Fat 8.0	ValueError / rejection	Rejected (ValueError)
T8	Non-numeric text	Input rejected and retried	Rejected and retried

Five-Entry Manual Total

Entry	Litres	Fat	SNF	Rate	Payment
1	10	3.5	8.5	Rs 30.00	Rs 300.00
2	8.5	4.2	8.7	Rs 33.40	Rs 283.90
3	12	3.1	8.3	Rs 27.80	Rs 333.60
4	5	3.6	8.5	Rs 30.40	Rs 152.00
5	4	3.5	8.4	Rs 29.70	Rs 118.80
Total					Rs 1188.30

Manual versus Automated Testing

Manual testing requires the tester to type each input and compare the output again after every change. Automated assert tests rerun the same cases consistently and quickly. Testing n cases takes O(n) time; the tests use O(n) space if all cases are held in a collection.


CSA0823---Python-Programing [Codespaces: obscure space zebra]
task12.py
task11.py
task8.py
task9.py
task10.py
C01 AT1 > task9.py
from decimal import Decimal, ROUND_HALF_UP
BASE_RATE = Decimal("30.00")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
def rate_per_litre(fat, snf):
"""Calculate the milk rate."""
fat_steps = round((fat - 3.5) / 0.1)
snf_steps = round((snf - 8.5) / 0.1)
rate = BASE_RATE
rate += Decimal(fat_steps) * FAT_STEP_VALUE
rate += Decimal(snf_steps) * SNF_STEP_VALUE
return rate.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
def payout(litres, fat, snf):
"""Calculate the payment."""
return (litres * rate_per_litre(fat, snf)).quantize(
Decimal("0.01"), rounding=ROUND_HALF_UP
Problems
Output
Debug Console
Terminal
Ports
bash - C01 AT1
@tharu-ui ➜/workspaces/CSA0823---Python-Programing/C01 AT1 (main) $ python task9.py
Expected: Rs. 300.00, Actual: Rs. 300.00, PASS
Expected: Rs. 283.90, Actual: Rs. 283.90, PASS
Expected: Rs. 333.60, Actual: Rs. 333.60, PASS
Expected: Rs. 120.00, Actual: Rs. 120.00, PASS
Expected: Rs. 220.00, Actual: Rs. 220.00, PASS
Five-entry total: Rs. 1257.50
Invalid-entry tests: PASS
Enter litres: 10
Enter fat percentage: 3.5
Enter SNF percentage: 2.5
Payment: Rs. 120.00
@tharu-ui ➜/workspaces/CSA0823---Python-Programing/C01 AT1 (main) $
Codespaces: obscure space zebra
main
Tharuniya (33 minutes ago)
Ln 12, Col 1
Spaces: 4
UTF-8
LF
Python
Sign In
Layout: US

TASK 10 – COMPARE EXPECTATIONS WITH EXECUTION (4 MARKS)

Benchmark Program

```
from decimal import Decimal
from time import perf_counter

SIZES = [300, 6000, 150000]
PAYMENT = "33.40"

def float_total(items):
    """Calculate a float total."""
    total = 0.0
    for amount in items:
        total += float(amount)
    return total

def decimal_total(items):
    """Calculate a Decimal total."""
    total = Decimal("0.00")
    for amount in items:
        total += Decimal(amount)
    return total

for n in SIZES:
    payments = [PAYMENT] * n
    start = perf_counter()
    f_total = float_total(payments)
    float_time = perf_counter() - start

    start = perf_counter()
    d_total = decimal_total(payments)
    decimal_time = perf_counter() - start
```

```
    difference = abs(Decimal(str(f_total)) - d_total)
    print(f"\nPayments: {n}")
    print(f"Float total:   {f_total:.10f}")
    print(f"Decimal total: {d_total:.2f}")
    print(f"Difference:    {difference}")
    print(f"Float time:    {float_time:.6f} seconds")
    print(f"Decimal time:  {decimal_time:.6f} seconds")
```

Expected Behaviour and Interpretation

Both methods process every payment, so the time complexity is O(n). The float result may contain a small representation error, while Decimal constructed from strings preserves the input decimal values. Decimal is often slower than float, but the actual timing depends on the computer, Python version and background activity.

Rounding Each Payment or Only the Total

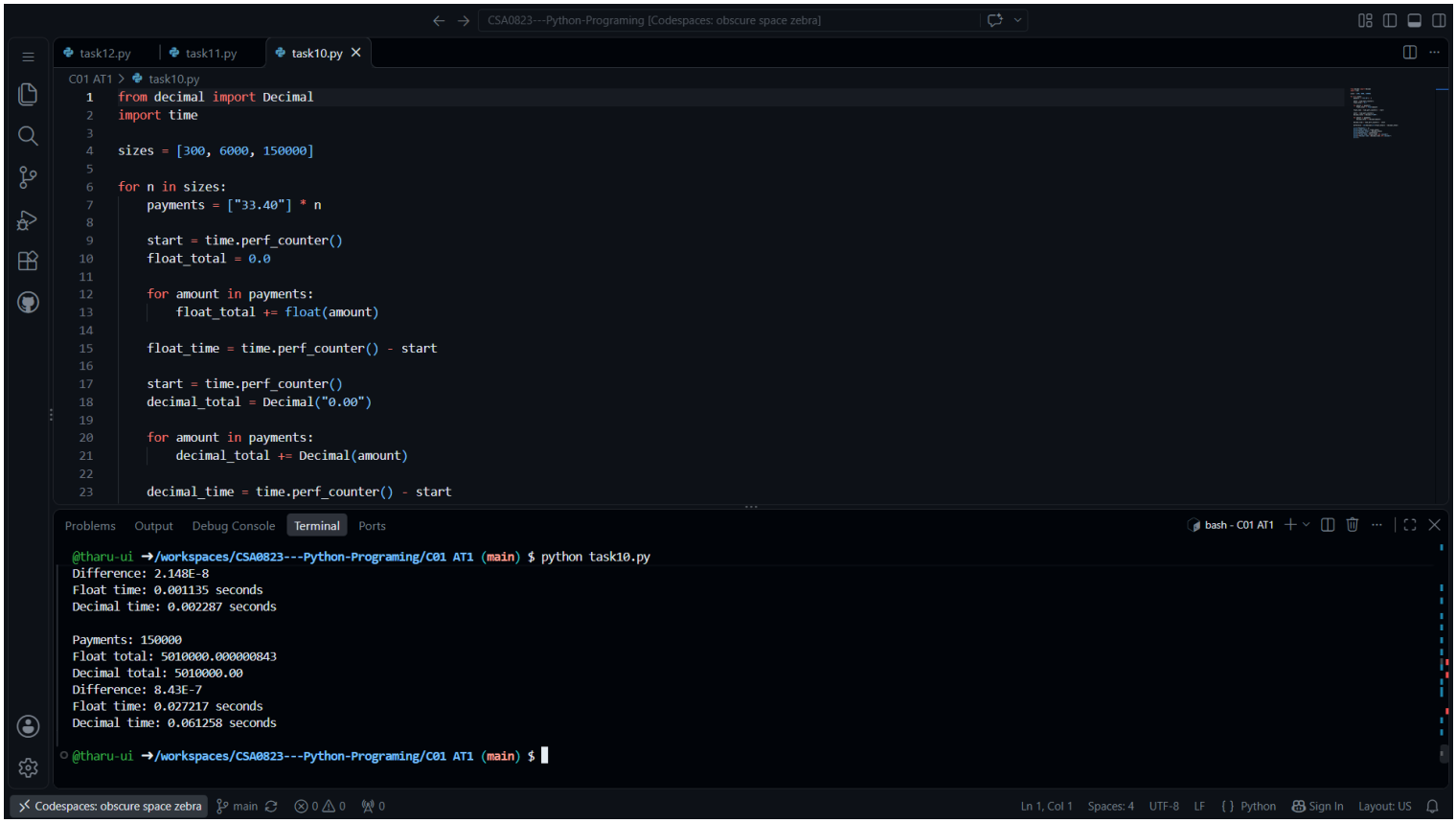
Rounding every payment is appropriate if the official business rule requires each payment to be rounded before recording. Rounding only the total may be appropriate if the rule specifies a single final rounding operation. The co-operative must define this rule. Decimal or integer paise avoids reliance on binary floating-point approximations.

Rate Cache Alternative

```
rate_cache = {}

def cached_rate(fat, snf):
    """Return a cached rate or calculate it once."""
    key = (fat, snf)
    if key not in rate_cache:
        rate_cache[key] = rate_per_litre(fat, snf)
    return rate_cache[key]
```

This example assumes rate_per_litre() has been imported or defined. Average dictionary lookup is O(1), and a cache for k distinct fat/SNF pairs uses O(k) space. Caching is most useful when many entries repeat the same pair. If most pairs are unique, the extra storage may not provide a benefit.



TASK 11 – RECOMMEND AND DEFEND THE FINAL DESIGN (4 MARKS)

Recommendation

I recommend a modular design with named constants, input validation, reusable functions and Decimal or integer paise for financial values. The final choice should be supported by the measurements collected in Task 10.

Decision	Justification
Named constants or a central rate card	A rate revision can be made in one place.
Functions and a separate module	Calculation logic can be reused and tested independently.
Input validation	Invalid quantities and measurements are rejected before payment calculation.
Decimal or integer paise	Avoids binary float representation errors in money totals.
Automated tests	Normal, boundary and invalid cases can be rerun after changes.

Rounding and Rate Revisions

Use decimal monetary arithmetic and an explicitly documented two-decimal rounding policy. The example code uses ROUND_HALF_UP; the co-operative must confirm that this matches its official policy. For a revision, update the rate card in one place and rerun the complete test suite.

Limitations and District-Union Scale

The initial design assumes a fixed rate card and step size. A larger district-union deployment would require a database, transaction identifiers, role-based access, audit records, backups, concurrency handling and centrally managed rate versions. The modular calculation design can be reused, but these operational features are needed for a reliable production system.

Alternative: Read a JSON Rate Card

{

```
  "base_rate": "30.00",
  "fat_step_value": "0.40",
  "snf_step_value": "0.30",
  "standard_fat": "3.5",
  "standard_snf": "8.5",
  "step_size": "0.1"
}
```

```
import json
from decimal import Decimal

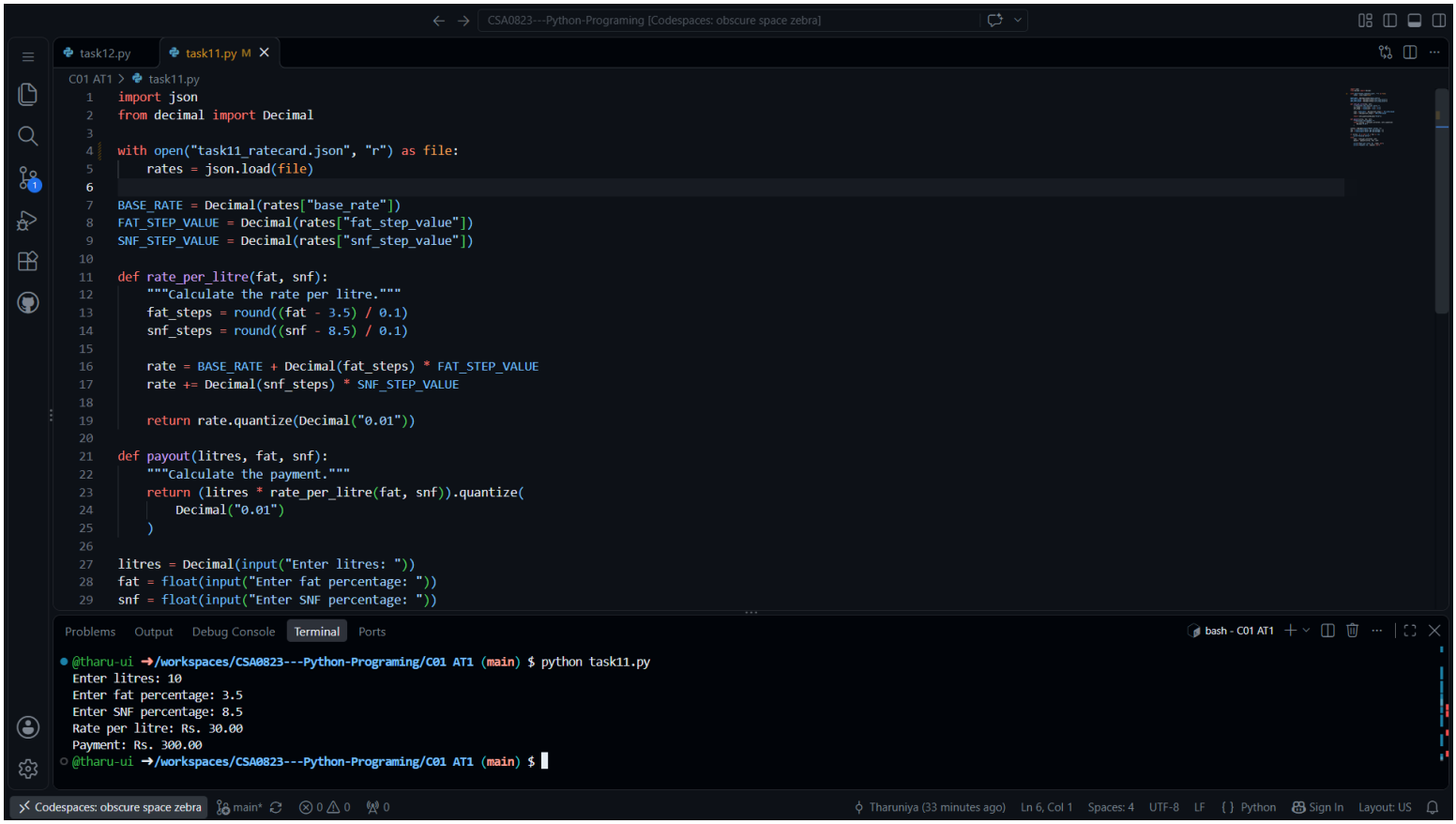
with open("rate_card.json", "r", encoding="utf-8") as file:
    data = json.load(file)

BASE_RATE = Decimal(data["base_rate"])
FAT_STEP_VALUE = Decimal(data["fat_step_value"])
SNF_STEP_VALUE = Decimal(data["snf_step_value"])
```

The remaining fields can be loaded similarly. Loading t configuration values takes O(t) time and O(t) space. It adds a small startup cost but allows rate changes without editing the calculation source code. The application should validate the file and protect it against unauthorized changes.

Evidence-Based Defence

In one benchmark run, the float and Decimal totals for 300 payments were 10019.999999999944 and 10020.00; for 6,000 payments, they were 200399.99999997852 and 200400.00; and for 150,000 payments, they were 5010000.000000843 and 5010000.00. For 150,000 payments, the measured times were approximately 0.011810 seconds for float and 0.034608 seconds for Decimal. The timings may vary by computer and execution conditions. I selected Decimal for monetary calculations because exact decimal representation and a consistent rounding policy are more important than the small performance advantage of float for this workload.


CSA0823---Python-Programing [Codespaces: obscure space zebra]
task12.py
task11.py M
C01 AT1 > task11.py
import json
from decimal import Decimal
with open("task11_ratecard.json", "r") as file:
rates = json.load(file)
BASE_RATE = Decimal(rates["base_rate"])
FAT_STEP_VALUE = Decimal(rates["fat_step_value"])
SNF_STEP_VALUE = Decimal(rates["snf_step_value"])
def rate_per_litre(fat, snf):
"""Calculate the rate per litre."""
fat_steps = round((fat - 3.5) / 0.1)
snf_steps = round((snf - 8.5) / 0.1)
rate = BASE_RATE + Decimal(fat_steps) * FAT_STEP_VALUE
rate += Decimal(snf_steps) * SNF_STEP_VALUE
return rate.quantize(Decimal("0.01"))
def payout(litres, fat, snf):
"""Calculate the payment."""
return (litres * rate_per_litre(fat, snf)).quantize(
Decimal("0.01")
)
litres = Decimal(input("Enter litres: "))
fat = float(input("Enter fat percentage: "))
snf = float(input("Enter SNF percentage: "))
Problems
Output
Debug Console
Terminal
Ports
bash - C01 AT1
@tharu-ui ➜/workspaces/CSA0823---Python-Programing/C01 AT1 (main) $ python task11.py
Enter litres: 10
Enter fat percentage: 3.5
Enter SNF percentage: 8.5
Rate per litre: Rs. 30.00
Payment: Rs. 300.00
@tharu-ui ➜/workspaces/CSA0823---Python-Programing/C01 AT1 (main) $
Codespaces: obscure space zebra
main*
Tharuniya (33 minutes ago)
Ln 6, Col 1
Spaces: 4
UTF-8
LF
Python
Sign In
Layout: US

TASK 12 – RE-EVALUATE AND ADAPT THE DESIGN (8 MARKS)

Changed Requirements

The analyser now supplies fat and SNF readings to two decimal places. From April to June, the base rate is Rs 32.00 and each fat step is worth Rs 0.45. Farmers who supply more than 500 litres in ten days receive a bonus of Rs 0.50 per litre.

A. Impact Analysis (2 Marks)

- The original fixed rate card must now support seasonal values.
- The base rate and fat-step value change in April, May and June; the standard fat, standard SNF, step size and SNF-step value are retained because the revised statement does not specify changes to them.
- Fat and SNF values with two decimal places require a defined rule for converting the measured difference into 0.1 percentage-point steps.
- A new ten-day total and bonus calculation is needed. The bonus applies only when the ten-day quantity is greater than 500 litres.

B. Evaluation of the Original Design (2 Marks)

The original constants, functions and module remain useful, but a single fixed rate card is insufficient. The rate calculation should accept the selected month or rate card, and a separate function should apply the volume bonus.

For fat 4.23 and SNF 8.67, the nearest-whole-step rule gives approximately 7 fat steps and 2 SNF steps. The original rate is Rs 30.00 + (7 × Rs 0.40) + (2 × Rs 0.30) = Rs 33.40. During April–June, the rate becomes Rs 32.00 + (7 × Rs 0.45) + (2 × Rs 0.30) = Rs 35.75.

For 510 litres, the bonus is 510 × Rs 0.50 = Rs 255.00, because the quantity is greater than 500 litres.

C. Revised Implementation (2 Marks)

```
from decimal import Decimal, ROUND_HALF_UP
```

```
STANDARD_FAT = Decimal("3.5")
STANDARD_SNF = Decimal("8.5")
STEP_SIZE = Decimal("0.1")
BONUS_LIMIT = Decimal("500")
BONUS_PER_LITRE = Decimal("0.50")

def money(value):
    """Round a monetary value to two decimal places."""
    return value.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)

def rate_card(month):
    """Return the rate card for a month."""
    if not 1 <= month <= 12:
        raise ValueError("Month must be between 1 and 12.")
    if 4 <= month <= 6:
        return {
            "base": Decimal("32.00"),
            "fat": Decimal("0.45"),
            "snf": Decimal("0.30")
        }
    return {
        "base": Decimal("30.00"),
        "fat": Decimal("0.40"),
        "snf": Decimal("0.30")
    }

def rate_per_litre(fat, snf, month):
    """Calculate the rate using the selected rate card."""
    card = rate_card(month)
    fat_steps = int(((fat - STANDARD_FAT) / STEP_SIZE).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP))
    snf_steps = int(((snf - STANDARD_SNF) / STEP_SIZE).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP))
```

```
    rate = card["base"] + fat_steps * card["fat"]
    rate += snf_steps * card["snf"]
    return money(rate)

def payout(litres, fat, snf, month):
    """Return the payment for one valid entry."""
    if litres <= 0:
        raise ValueError("Litres must be positive.")
    if not Decimal("2.0") <= fat <= Decimal("7.0"):
        raise ValueError("Fat is outside the allowed range.")
    if not Decimal("0.0") <= snf <= Decimal("12.0"):
        raise ValueError("SNF is outside the allowed range.")
    return money(litres * rate_per_litre(fat, snf, month))

def ten_day_total(entries, month):
    """Return normal payment, bonus and final total."""
    total_litres = sum((e[0] for e in entries), Decimal("0"))
    normal = sum((payout(litres, fat, snf, month)
                  for litres, fat, snf in entries), Decimal("0.00"))
    bonus = money(total_litres * BONUS_PER_LITRE) if total_litres > BONUS_LIMIT else
Decimal("0.00")
    return money(normal), bonus, money(normal + bonus)

if __name__ == "__main__":
    entries = [
        (Decimal("51"), Decimal("4.23"), Decimal("8.67"))
        for _ in range(10)
    ]
    for month in [3, 4]:
        normal, bonus, total = ten_day_total(entries, month)
        print("Month:", month)
        print("Rate per litre:",
              rate_per_litre(Decimal("4.23"), Decimal("8.67"), month))
```

```
print("Normal payment:", normal)
print("Bonus:", bonus)
print("Total payment:", total)
```

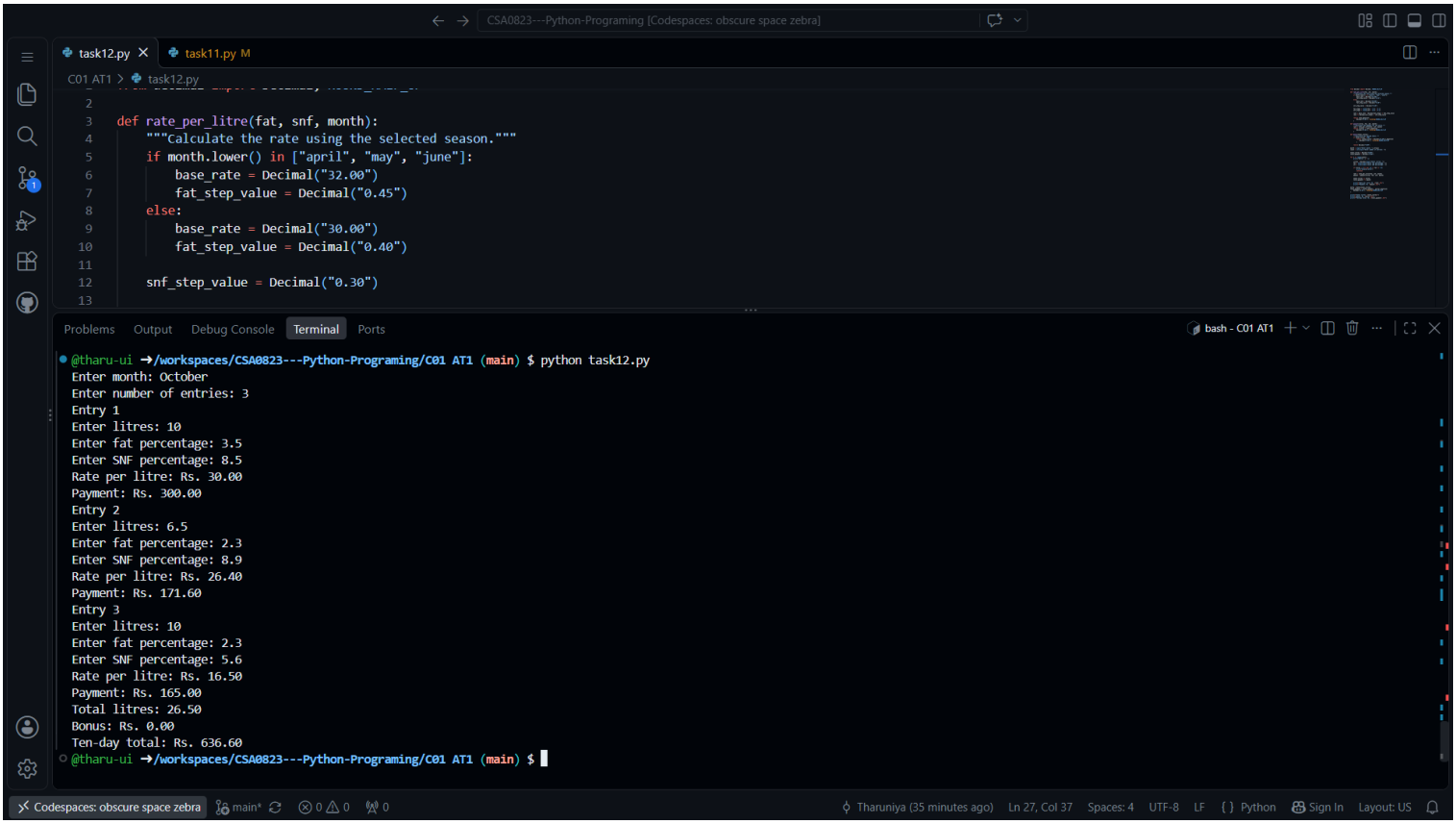
Expected Results for the Worked Example

Measure	March (original rates)	April (seasonal rates)
Rate per litre	Rs 33.40	Rs 35.75
Ten-day quantity	510 L	510 L
Normal payment	Rs 17034.00	Rs 18232.50
Bonus	Rs 255.00	Rs 255.00
Total payment	Rs 17289.00	Rs 18487.50

D. Evidence-Based Defence (2 Marks)

I modified the original design instead of replacing it completely. The existing functions and module structure remain useful, while the rate-card function selects the appropriate seasonal values. Decimal arithmetic supports the decimal inputs and monetary calculations. The worked example changes from Rs 33.40 to Rs 35.75 per litre when the month changes from March to April. At 510 litres, the bonus is Rs 255.00 in both cases. This is easier to maintain than separate programs for each season.

This implementation applies the Rs 0.50 bonus to all litres once the ten-day total exceeds 500 litres and selects the rate card by month. These interpretations should be checked against the complete assessment statement.


CSA0823---Python-Programing [Codespaces: obscure space zebra]
task12.py
task11.py M
C01 AT1 > task12.py
def rate_per_litre(fat, snf, month):
"""Calculate the rate using the selected season."""
if month.lower() in ["april", "may", "june"]:
base_rate = Decimal("32.00")
fat_step_value = Decimal("0.45")
else:
base_rate = Decimal("30.00")
fat_step_value = Decimal("0.40")
snf_step_value = Decimal("0.30")
Problems
Output
Debug Console
Terminal
Ports
bash - C01 AT1
@tharu-ui ➜/workspaces/CSA0823---Python-Programing/C01 AT1 (main) $ python task12.py
Enter month: October
Enter number of entries: 3
Entry 1
Enter litres: 10
Enter fat percentage: 3.5
Enter SNF percentage: 8.5
Rate per litre: Rs. 30.00
Payment: Rs. 300.00
Entry 2
Enter litres: 6.5
Enter fat percentage: 2.3
Enter SNF percentage: 8.9
Rate per litre: Rs. 26.40
Payment: Rs. 171.60
Entry 3
Enter litres: 10
Enter fat percentage: 2.3
Enter SNF percentage: 5.6
Rate per litre: Rs. 16.50
Payment: Rs. 165.00
Total litres: 26.50
Bonus: Rs. 0.00
Ten-day total: Rs. 636.60
@tharu-ui ➜/workspaces/CSA0823---Python-Programing/C01 AT1 (main) $
Codespaces: obscure space zebra
main*
Tharuniya (35 minutes ago)
Ln 27, Col 37
Spaces: 4
UTF-8
LF
Python
Sign In
Layout: US